PROJECT: HearMeAI (Speech AI System)

1. What We Built (Detailed Explanation)

We developed a web-based Speech AI system that processes audio input and performs two main tasks:

1. **Converts spoken audio into text using Automatic Speech Recognition (ASR)**
2. **Identifies and separates different speakers in the conversation using speaker diarization**

The system is designed to handle real-world audio inputs and present results in a structured and user-friendly format.

Key Features:

- **Supports audio input through file upload and live microphone recording**
- **Converts speech into accurate, timestamped text**
- **Identifies multiple speakers and assigns labels (Speaker 1, Speaker 2, etc.)**
- **Displays transcripts with color-coded speakers for clarity**
- **Provides speaker-wise statistics such as speaking duration**
- **Allows users to download the final transcript as a text file**

---

2. Overall Workflow (System Pipeline)

Audio Input → Preprocessing → ASR (Whisper) → Speaker Diarization → Merge → UI Output

Step-by-step Explanation:

1. **Audio Input**
   The system accepts audio either from uploaded files (mp3, wav, etc.) or directly from the microphone.
2. **Preprocessing**
   The input audio is converted into a standard format (16 kHz mono WAV).
   This ensures compatibility and improves model performance.
3. **Speech-to-Text (ASR)**
   The processed audio is passed to the Whisper model, which generates:
   - **Transcribed text**
   - **Timestamps for each segment**

    - **Detected language**

4. **Speaker Diarization**
   The same audio is analyzed to identify different speakers by:
    - **Extracting audio features (MFCCs)**
    - **Grouping similar voice patterns using clustering**
5. **Merging Results**
   The system aligns transcription segments with speaker segments using timestamps to assign the correct speaker to each text segment.
6. **UI Output**
   The final output is displayed with:
    - **Colored speaker labels**
    - **Time-aligned transcript**
    - **Speaker statistics**

---

4. Technologies and Tools Used

Programming Language:

- **Python**

AI and Machine Learning:

- **Whisper (ASR model for transcription)**
- **K-Means clustering (for speaker grouping)**

Libraries:

- **PyTorch: backend framework used by Whisper**
- **Librosa: audio loading and feature extraction (MFCC)**
- **NumPy: numerical operations**
- **Scikit-learn: clustering (KMeans) and evaluation (silhouette score)**
- **Streamlit: web interface development**

---

4. Why We Chose These Technologies

Whisper (ASR)

- **Provides high-quality transcription**
- **Works without internet (offline inference)**
- **Supports multiple languages**
- **Pre-trained model reduces development time**

Librosa

- **Widely used in audio processing**
- **Simplifies feature extraction**
- **Efficient and reliable**

K-Means Clustering

- **Simple and efficient algorithm**
- **Does not require labeled data**
- **Suitable for quick implementation in hackathon settings**

Streamlit

- **Enables rapid UI development**
- **No need for frontend frameworks**
- **Easy to deploy and test**

---

5. Alternatives and Why We Did Not Use Them

ASR Alternatives

- **Google Speech API: requires API key and has usage costs**
- **Azure Speech Services: paid service with dependency on cloud**
- **DeepSpeech: lower accuracy compared to Whisper**
- **Wav2Vec2: requires more setup and tuning**

Speaker Diarization Alternatives

- **Pyannote.audio: highly accurate but requires GPU and complex setup**
- **NVIDIA NeMo: heavy framework and longer setup time**
- **SpeechBrain: more suited for research-level implementations**

Reason for choosing K-Means:
It is lightweight, fast, and sufficient for demonstrating core concepts.

UI Alternatives

- **React with backend (Flask/FastAPI): requires more time and frontend skills**
- **Flask alone: lacks built-in UI components**

Reason for choosing Streamlit:
It allows fast development with minimal effort.

---

6. What We Built (Final Deliverables)

- **Fully functional web application**
- **Supports both file-based and live audio input**
- **Multi-speaker transcription system**
- **Color-coded transcript visualization**
- **Speaker-wise time statistics**
- **Downloadable transcript output**

---

7. Concepts Used

- **Automatic Speech Recognition (ASR)**
- **Audio signal processing**
- **Feature extraction using MFCC**
- **Machine Learning (Clustering)**
- **Deep Learning (Whisper model)**
- **Data processing and visualization**

---

8. Limitations of Current System

- **Diarization accuracy depends on audio quality**
- **K-Means is not as accurate as deep learning models**
- **Does not support overlapping speech well**
- **No real-time streaming transcription**

---

9. What We Did Not Implement

- **Text-to-Speech (TTS)**
- **Voice cloning**
- **Voice conversion**
- **Training custom deep learning models**

However, Whisper internally uses Transformer-based deep learning architecture.

---

10. Common Questions and Answers

Q: Where did you use PyTorch?
Answer: Whisper is built using PyTorch, and it is used internally during transcription.

Q: How do you detect speakers?
Answer: We extract MFCC features from the audio and apply K-Means clustering to group similar voice characteristics.

Q: How do you decide the number of speakers?
Answer: We use silhouette score to automatically estimate the optimal number of clusters.

Q: Why did you not use deep learning for diarization?
Answer: Due to time and resource constraints, we used a lightweight machine learning approach.

---

11. Final Summary

We developed a Speech AI system that takes audio input, processes it, converts speech into text using Whisper, identifies speakers using MFCC features and K-Means clustering, and presents the results in an interactive web interface built with Streamlit.